

=> Update on Algorithm (parse tree => human-readable query)

First, I manually analyzed the complete logs to think about possible approaches for solving this problem. The first thing I observed was that the parse tree contains the table and column names as they appear in the query.

Parsing phase tree:

```
:whereClause
{BOOLEXPR
:boolop and
:args (
{A_EXPR
:name (">=")
:lexpr
{COLUMNREF
:fields ("o_orderdate")
:location 118
}
}
:rexpr
{TYPECAST
:arg
{A_CONST
:val "\1994-01-01"
:location 138
}
:typeName
{TYPPNAME
:names ("date")
:typeOid 0
:setof false
:pct_type false
:typmods <>
:typemod -1
:arrayBounds <>
:location 133
}
}
:location -1
}
:rexpr_list_start 0
:rexpr_list_end 0
:location 130
}
```

After the parsing stage, PostgreSQL replaces the column names with internal numeric identifiers. For example, it represents columns using numbers instead of their original names. Therefore, without additional context about the tables, we cannot interpret the parse tree and obtain meaningful results from it alone.

Analyzer phase tree structure:

```
{OPEXPR
:opno 2345
:opfuncid 2338
:opresulttype 16
:opretset false
:opcollid 0
:inputcollid 0
:args (
{VAR
:varno 1
:varattno 5
:varatype 1082
:varitypmid -1
:varcollid 0
:varnullingrela (b)
:varlevelsup 0
:varreturningtype 0
:varnosyn 1
:varattnosyn 5
:location 159
}
)
{OPEXPR
:opno 1076
:opfuncid 2071
:opresulttype 1114
:opretset false
:opcollid 0
:inputcollid 0
:args (
{CONST
:consttype 1082
:consttypmid -1
:constcollid 0
:constlen 4
:constbyval true
:constisnull false
:location 178
:constvalue 4 [ 113 -9 -1 -1 -1 -1 -1 ]
}
{CONST
:consttype 1186
:consttypmid 196607
:constcollid 0
:constlen 16
:constbyval false
:constisnull false
:location 202
:constvalue 16 [ 0 0 0 0 0 0 0 0 0 0 0 0 3 0 0 0 ]
}
)
}
:location 191
}
```

In the left image, securityQuals contains the RLS policy condition, whereas in the right image, the query condition is stored in the jointree->quals field.

[illegible]

For views, the view condition is stored inside the SUBLINK under RTE_SUBQUERY->subquery->jointree->quals. This happens because the rewriter expands the view definition into a subquery. As a result, the view predicate appears in the quals field of that subquery.

```
:quals
{SUBLINK
:sublinkType 0
:sublinkId 0
:testexpr <>
:opName <>
:subselect
{QUERY
:commandType 1
:querySource 0
:canSetTag true
:utilityStmt <>
:resultRelation 0
:hasAggs false
:hasWindowFuncs false
:hasTargetSRFs false
:hasSublinks false
:hasDistinctOn false
:hasRecursive false
:hasModifyingCTE false
:hasForUpdate false
:hasRowSecurity false
:hasGroupRTE false
:isReturn false
:ctelist <>
:table (
{RANGETBLENTY
:alias
{ALIAS
:aliasname c
:colnames <>
}
:eref
{ALIAS
:aliasname c
:colnames ("c_custkey" "c_name" "c_address" "c_nationk
ey" "c_phone" "c_acctbal" "c_mktsegment" "c_comment")
}
:cteKind 0
:relid 16546
:inh true
:relkind r
:rellockmode 1
:permInfoIndex 1
:tablesample <>
:lateral false
:inFromCl true
:securityQuals <>
}
)
:ctePerminfos (
{RTEPERMISSIONINFO
:relid 16546
:inh true
:requiredPerms 2
:checkAsUser 10
:selectedCols (b 8 13)
:insertedCols (b)
:updatedCols (b)
}
)
:jointree
{FROMEXPR
:fromlist (
{RANGETBLREF
:ctindex 1
}
)
:quals
{BOOLEXP
:boolop and
:args (
{OPEXP
:opno 96
:opfuncid 65
:opresulttype 16

```

So, instead of spending time developing a universal algorithm to convert PostgreSQL logs into human-readable SQL, I chose to reduce the number of logs being generated. The reason is that PostgreSQL uses different internal structures at different stages of query processing, and the format of the logs depends on where they are printed. Therefore, building a generic log converter would be very time-consuming.

By reducing the logs, it became easier to manually trace the query transformations through each stage.

The biggest task now is to understand the query transformations that occur inside the planner.

Last Friday, we discussed an important question: if both RLS predicates and View predicates eventually become similar restriction clauses after security-level processing, why is there still a significant difference in execution time?

The planner treats the optimization of securityQuals and jointree->quals differently. To understand the complete process, I need to perform debugging at the planner level.

As of now, we can use this EXPLAIN to view the final plans for both Views and RLS. I have also added logs for the planner's proposed plan and the executor's executed plan to better understand the final plan and any differences between the proposed and executed plans.

EXPLAIN (

ANALYZE true,
VERBOSE true,
COSTS true,
BUFFERS true,
TIMING true,
FORMAT JSON

) query

RLS:

```
WARNING: === PLANNER SUGGESTION ===
Plan:
GroupAggregate (cost=8883274415.71..8883274719.76 rows=5 width=17)
  Output: orders.o_orderpriority, count(*)
  Group Key: orders.o_orderpriority
  -> Sort (cost=8883274415.71..8883274517.04 rows=40534 width=9)
    Output: orders.o_orderpriority
    Sort Key: orders.o_orderpriority
    -> Hash Semi Join (cost=212549.34..8883271313.47 rows=48534 width=9)
      Output: orders.o_orderpriority
      Hash Cond: (orders.o_orderkey = lineitem.l_orderkey)
      -> Seq Scan on public.orders (cost=0.00..8883048568.00 rows=144199 width=13)
        Output: orders.o_orderpriority, orders.o_orderkey
        Filter: ((orders.o_orderdate >= '1994-01-01'::date) AND (ANY (orders.o_custkey = (hashed SubPlan 2).col1)) AND (orders.o_orderdate < '1994-04-01 00:00:00'::timestamp without time zone))
      e))
        SubPlan 2
        -> Seq Scan on public.customer c (cost=0.00..5547.00 rows=67888 width=4)
          Output: c.c_custkey
          Filter: (c.c_acctbal > '5000'::double precision)
          -> Hash (cost=179731.50..179731.50 rows=2000307 width=4)
            Output: lineitem.l_orderkey
            -> Seq Scan on public.lineitem (cost=0.00..179731.50 rows=2000307 width=4)
              Output: lineitem.l_orderkey
              Filter: (lineitem.l_commitdate < lineitem.l_receiptdate)
            e))
    e))
=====
WARNING: === EXECUTOR EXECUTION ===
Rows Processed: 5
Plan & Stats:
GroupAggregate (cost=8883274415.71..8883274719.76 rows=5 width=17) (actual time=2820.262..2822.023 rows=5.00 loops=1)
  Output: orders.o_orderpriority, count(*)
  Group Key: orders.o_orderpriority
  Buffers: shared hit=4803 read=129600, temp read=14748 written=14748
  -> Sort (cost=8883274415.71..8883274517.04 rows=40534 width=9) (actual time=2819.809..2820.637 rows=23317.00 loops=1)
    Output: orders.o_orderpriority
    Sort Key: orders.o_orderpriority
    Sort Method: quicksort Memory: 769kB
    Buffers: shared hit=4803 read=129600, temp read=14748 written=14748
    -> Hash Semi Join (cost=212549.34..8883271313.47 rows=48534 width=9) (actual time=1943.572..2812.812 rows=23317.00 loops=1)
      Output: orders.o_orderpriority
      Hash Cond: (orders.o_orderkey = lineitem.l_orderkey)
      Buffers: shared hit=4800 read=129600, temp read=14748 written=14748
      -> Seq Scan on public.orders (cost=0.00..8883048568.00 rows=144199 width=13) (actual time=202.119..615.014 rows=25383.00 loops=1)
        Output: orders.o_orderpriority, orders.o_orderkey
        Filter: ((orders.o_orderdate >= '1994-01-01'::date) AND (ANY (orders.o_custkey = (hashed SubPlan 2).col1)) AND (orders.o_orderdate < '1994-04-01 00:00:00'::timestamp without time zone))
        e))
      Rows Removed by Filter: 1474617
      Buffers: shared hit=4236 read=25444
      SubPlan 2
      -> Seq Scan on public.customer c (cost=0.00..5547.00 rows=67888 width=4) (actual time=0.299..183.842 rows=67989.00 loops=1)
        Output: c.c_custkey
        Filter: (c.c_acctbal > '5000'::double precision)
        Rows Removed by Filter: 82011
        Buffers: shared hit=3672
      -> Hash (cost=179731.50..179731.50 rows=2000307 width=4) (actual time=1736.597..1736.597 rows=3793296.00 loops=1)
        Output: lineitem.l_orderkey
        Buckets: 262144 (originally 262144) Batches: 32 (originally 16) Memory Usage: 6223kB
        Buffers: shared hit=804 read=104156, temp written=10750
        -> Seq Scan on public.lineitem (cost=0.00..179731.50 rows=2000307 width=4) (actual time=0.513..1141.892 rows=3793296.00 loops=1)
          Output: lineitem.l_orderkey
          Filter: (lineitem.l_commitdate < lineitem.l_receiptdate)
          Rows Removed by Filter: 2207919
          Buffers: shared hit=804 read=104156
        e))
    e))
=====
```

The good thing is that this format is human-readable, so we can clearly see the differences.

```

WARNING: === PLANNER SUGGESTION ===
Plan:
Finalize GroupAggregate (cost=194594.52..194617.59 rows=5 width=17)
  Output: o.o.orderpriority, count(*)
  Group Key: o.o.orderpriority
  -> Gather Merge (cost=194594.52..194617.48 rows=12 width=17)
    Output: o.o.orderpriority, (PARTIAL count(*))
    Workers Planned: 2
    -> Partial GroupAggregate (cost=193594.49..193616.07 rows=5 width=17)
      Output: o.o.orderpriority, PARTIAL count(*)
      Group Key: o.o.orderpriority
      -> Sort (cost=193594.49..193601.67 rows=2878 width=9)
        Output: o.o.orderpriority
        Sort Key: o.o.orderpriority
        -> Parallel Hash Semi Join (cost=154485.47..193429.66 rows=2878 width=9)
          Hash Cond: (o.o.orderkey = lineitem.l_orderkey)
          -> Parallel Hash Semi Join (cost=4886.41..48362.28 rows=18218 width=13)
            Output: o.o.orderpriority, o.o.orderkey
            Hash Cond: (o.o.custkey = c.c.custkey)
            -> Parallel Seq Scan on public.orders o (cost=0.00..35383.00 rows=22585 width=17)
              Output: o.o.orderkey, o.o.custkey, o.o.orderstatus, o.o.totalprice, o.o.orderdate, o.o.orderpriority, o.o.clerk, o.o.shippriority, o.o.comment
              Filter: ((o.o.orderdate >= '1994-01-01':date) AND (o.o.orderdate < '1994-04-01 00:00:00':timestamp without time zone))
            -> Parallel Hash (cost=4453.25..4453.25 rows=28253 width=4)
              Output: c.c.custkey
              -> Parallel Seq Scan on public.customer c (cost=0.00..4453.25 rows=28253 width=4)
                Output: c.c.custkey
                Filter: (c.c.acctbal > '5000':double precision)
              -> Parallel Hash (cost=135974.79..135974.79 rows=833461 width=4)
                Output: lineitem.l_orderkey
                -> Parallel Seq Scan on public.lineitem (cost=0.00..135974.79 rows=833461 width=4)
                  Output: lineitem.l_orderkey
                  Filter: (lineitem.l_commitdate < lineitem.l_receiptdate)

=====
WARNING: === EXECUTOR EXECUTION ===
Rows Processed: 5
Plan & Stats:
Finalize GroupAggregate (cost=194594.52..194617.59 rows=5 width=17) (actual time=1261.130..1285.725 rows=5.00 loops=1)
  Output: o.o.orderpriority, count(*)
  Group Key: o.o.orderpriority
  Buffers: shared hit=5084 read=129412, temp read=28143 written=28540
  -> Gather Merge (cost=194594.52..194617.48 rows=12 width=17) (actual time=1260.981..1285.714 rows=15.00 loops=1)
    Output: o.o.orderpriority, (PARTIAL count(*))
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=5084 read=129412, temp read=28143 written=28540
    -> Partial GroupAggregate (cost=193594.49..193616.07 rows=5 width=17) (actual time=1254.288..1254.984 rows=5.00 loops=3)
      Output: o.o.orderpriority, PARTIAL count(*)
      Group Key: o.o.orderpriority
      Buffers: shared hit=5084 read=129412, temp read=28143 written=28540
      Worker 0: actual time=1251.562..1252.288 rows=5.00 loops=1
        Buffers: shared hit=1676 read=43422, temp read=6746 written=6896
      Worker 1: actual time=1251.882..1252.397 rows=5.00 loops=1
        Buffers: shared hit=1674 read=43446, temp read=6715 written=6876
      -> Sort (cost=193594.49..193601.67 rows=2878 width=9) (actual time=1254.123..1254.407 rows=7772.33 loops=3)
        Output: o.o.orderpriority
        Sort Key: o.o.orderpriority
        Sort Method: quicksort Memory: 193kB
        Buffers: shared hit=5084 read=129412, temp read=28143 written=28540
        Worker 0: actual time=1251.392..1251.696 rows=8314.00 loops=1
          Sort Method: quicksort Memory: 385kB
          Buffers: shared hit=1676 read=43422, temp read=6746 written=6896
        Worker 1: actual time=1251.646..1251.922 rows=7707.00 loops=1
          Sort Method: quicksort Memory: 193kB
          Buffers: shared hit=1674 read=43446, temp read=6715 written=6876
        -> Parallel Hash Semi Join (cost=154485.47..193429.66 rows=2878 width=9) (actual time=1122.661..1251.924 rows=7772.33 loops=3)
          Output: o.o.orderpriority
          Hash Cond: (o.o.orderkey = lineitem.l_orderkey)
          Buffers: shared hit=4988 read=129412, temp read=28143 written=28540
          Worker 0: actual time=1111.232..1249.140 rows=8314.00 loops=1
            Buffers: shared hit=1668 read=43422, temp read=6746 written=6896
          Worker 1: actual time=1124.688..1249.335 rows=7707.00 loops=1
            Buffers: shared hit=1666 read=43446, temp read=6715 written=6876
          -> Parallel Hash Semi Join (cost=4886.41..48362.28 rows=18218 width=13) (actual time=14.845..154.297 rows=8461.00 loops=3)
            Output: o.o.orderpriority, o.o.orderkey
            Hash Cond: (o.o.custkey = c.c.custkey)
            Buffers: shared hit=4338 read=28350
            Worker 0: actual time=15.138..154.291 rows=8421.00 loops=1
              Buffers: shared hit=1456 read=8432
            Worker 1: actual time=15.146..154.257 rows=8538.00 loops=1
              Buffers: shared hit=1448 read=8454
            -> Parallel Seq Scan on public.orders o (cost=0.00..35383.00 rows=22585 width=17) (actual time=0.712..133.538 rows=18783.00 loops=3)
              Output: o.o.orderkey, o.o.custkey, o.o.orderstatus, o.o.totalprice, o.o.orderdate, o.o.orderpriority, o.o.clerk, o.o.shippriority, o.o.comment
              Filter: ((o.o.orderdate >= '1994-01-01':date) AND (o.o.orderdate < '1994-04-01 00:00:00':timestamp without time zone))
              Rows Removed by Filter: 481217
              Buffers: shared hit=608 read=28350
              Worker 0: actual time=0.821..133.891 rows=18754.00 loops=1
                Buffers: shared hit=232 read=8432
              Worker 1: actual time=1.099..133.562 rows=18665.00 loops=1
                Buffers: shared hit=215 read=8454
            -> Parallel Hash (cost=4453.25..4453.25 rows=28253 width=4) (actual time=13.930..13.930 rows=22663.00 loops=3)
              Output: c.c.custkey
              Buckets: 131872 Batches: 1 Memory Usage: 3712kB
              Buffers: shared hit=3672
              Worker 0: actual time=13.889..13.889 rows=22555.00 loops=1
                Buffers: shared hit=1224
              Worker 1: actual time=14.086..14.086 rows=22813.00 loops=1
                Buffers: shared hit=1225
              -> Parallel Seq Scan on public.customer c (cost=0.00..4453.25 rows=28253 width=4) (actual time=0.828..10.886 rows=22663.00 loops=3)
                Output: c.c.custkey
                Filter: (c.c.acctbal > '5000':double precision)
                Rows Removed by Filter: 27337
                Buffers: shared hit=3672
                Worker 0: actual time=0.816..10.859 rows=22555.00 loops=1
                  Buffers: shared hit=1224
                Worker 1: actual time=0.825..10.763 rows=22813.00 loops=1
                  Buffers: shared hit=1225
              -> Parallel Hash (cost=135974.79..135974.79 rows=833461 width=4) (actual time=949.561..949.561 rows=1264432.00 loops=3)
                Output: lineitem.l_orderkey
                Buckets: 262144 (originally 262144) Batches: 32 (originally 16) Memory Usage: 6782kB
                Buffers: shared hit=608 read=184862, temp read=9139 written=28156
                Worker 0: actual time=947.894..947.894 rows=1276844.00 loops=1
                  Buffers: shared hit=212 read=34998, temp read=3869 written=6768
                Worker 1: actual time=947.883..947.884 rows=1275478.00 loops=1
                  Buffers: shared hit=226 read=34992, temp read=3839 written=6748
                -> Parallel Seq Scan on public.lineitem (cost=0.00..135974.79 rows=833461 width=4) (actual time=1.328..541.725 rows=1264432.00 loops=3)
                  Output: lineitem.l_orderkey
                  Filter: (lineitem.l_commitdate < lineitem.l_receiptdate)
                  Rows Removed by Filter: 735973
                  Buffers: shared hit=658 read=184862
                  Worker 0: actual time=1.663..543.627 rows=1276844.00 loops=1
                    Buffers: shared hit=212 read=34998
                  Worker 1: actual time=1.492..550.191 rows=1275478.00 loops=1
                    Buffers: shared hit=226 read=34992
=====

```

I spent some time tracking the planner code on Friday and found that the planner treats securityQuals and a View's subquery differently. There may be cases where they eventually result in the same plan, but before reaching that conclusion, I need to complete the following tasks:

Trace both queries from the planner's starting point to the executor to fully understand the query transformation process. This time, I will analyze 3–4 queries to cover different scenarios and gain a broader understanding.

Investigate parallelization, which I still believe is one of the key reasons why Views are faster than RLS. I need to debug the planner code responsible for assigning the parallel_safe flag to each node so that we can understand how specific quals become eligible for parallel execution.

So, my next step is to thoroughly debug the planner section and understand the complete transformation process.